

Project Handover — AMW Annual Performance Evaluation System

Project	AMW Annual Performance Evaluation System
Version delivered	v1.4.6
Repository	https://github.com/Madebynoppawit/Performance-evaluation-Form
Handover date	2026-06-29
Status	✅ Demo-ready (spec-complete, all tests green). Production-ready pending customer infra (secrets + server).

1. What is delivered

A production-grade, full-stack annual performance-evaluation platform:

- Position-specific evaluation form (10 sections) with a unified **1–5 scoring** model.
- Goal setting, competency (dual manager/self), attendance, comments, salary, and signatures.
- Role-based access (Developer / Admin / Manager / Supervisor / Employee).
- Admin/HR calibration workspace, cycles, reports/BI, audit trail.
- Premium **PDF** (Thai+Latin) and **CSV** export.
- Bulk **employee import from a WinDev CSV export**; identity-based password reset.
- Tri-lingual UI (Thai / English / French).

Full feature/architecture overview: [README](#).

2. Deliverables checklist

Deliverable	Where	Status
Frontend source (React + TS)	<code>frontend/</code>	✓
Backend source (Express + Prisma)	<code>backend/</code>	✓
Database schema + migrations	<code>backend/prisma/</code>	✓
Synthetic demo seed	<code>backend/prisma/seed.cjs</code>	✓
Automated tests (360) + QA Robot (139)	<code>*/src/**</code> , <code>scripts/qa-robot/</code>	✓ all green
CI/CD pipelines	<code>.github/workflows/</code>	✓ green
Docker images + compose (dev/demo/prod/monitor)	<code>Dockerfile</code> , <code>docker-compose.*.yaml</code>	✓ build clean, 0 high/critical
Developer & Admin guide	<code>docs/developer-guide.md</code>	✓
API reference / data model	<code>docs/api.md</code> , <code>docs/data-model.md</code>	✓
Deploy checklist / ops runbook	<code>docs/deploy-checklist.md</code> , <code>docs/operations-runbook.md</code>	✓
Test catalog / threat model	<code>docs/system-test-cases.md</code> , <code>docs/threat-model.md</code>	✓
README + overview infographic	<code>README.md</code> , <code>docs/screenshots/</code>	✓

3. Acceptance status (as delivered)

- **Tests:** backend 41 unit + 139 integration, frontend 21 unit + 20 E2E, **QA Robot 139 — 360+ all green, 0 failures.**
- **CI on main** : green across lint, unit, integration, E2E, build, secret scan, and Docker image build + Trivy vuln-scan + push **(8/8)**.
- **Security:** no real employee data or secrets in git (verified by scan); production fails fast on weak JWT secret / wildcard CORS / missing metrics token.
- **Spec:** the evaluation form, rating scale (1–5 with behavior indicators), attendance thresholds, competencies, and 1–5 summary match the HR requirement.

4. Repository & access

- **Code & history:** the GitHub repository above (all work committed and pushed).
- **Releases:** tagged on GitHub; latest is `v1.4.6`.
- **Demo login IDs/passwords: not in the repo** — read from `backend/prisma/seed.cjs`. Rotate before any shared rollout.
- **Secrets** (`JWT_SECRET`, DB password, `METRICS_TOKEN`, SSH deploy keys): **not in the repo** — to be created by the customer in `.env.prod` and GitHub repo/Actions secrets.

5. Environments

Env	Compose file	Purpose
Development	<code>docker compose up -d postgres</code> + <code>npm run dev</code>	local coding
Demo	<code>docker-compose.demo.yml</code>	safe showcase (synthetic data)
Production	<code>docker-compose.prod.yml</code>	live (patched, scanned images)
Monitoring	<code>docker-compose.monitor.yml</code>	Prometheus/Grafana stack

Run/setup steps: [Developer & Admin Guide §2](#) and [deploy-checklist.md](#).

6. Data & security notes

- **Employee data source:** company **WinDev** → export CSV → import via the app's User/Data Management page (`employeeImportService` upserts into PostgreSQL). The exported CSV is real PII — **keep it out of the repo**; it is never committed.
- Real data lives only in the PostgreSQL database (runtime). Back it up via `backup.yml` / `BACKUP_DIR` (see the runbook), **not** by saving files in the repo.
- Password reset is identity-based (employee number + date of birth), not email.

7. Outstanding items (customer responsibility, ~5% to go-live)

1. Provision a production server and PostgreSQL; fill `.env.prod` with **strong** secrets (`openssl rand -base64 48` for `JWT_SECRET` , DB password, `METRICS_TOKEN`).
2. Add GitHub Actions secrets (`DEPLOY_HOST` , `DEPLOY_USER` , `DEPLOY_SSH_KEY`) and set the repo variable `DEPLOY_ENABLED=true` to enable auto-deploy (it safely skips until then).
3. Point a domain + HTTPS at the frontend; configure `CLIENT_URL` / `CORS_ORIGINS` .
4. Import the real employee roster from WinDev (CSV) and rotate demo credentials.
5. Run the post-deploy smoke test in [deploy-checklist.md](#).

No known functional defects are outstanding at handover.

8. Maintenance & support recommendations

- Keep CI green; run `npm run verify` before every change; branch off `main` and PR.
- Apply base-image patches periodically (the Dockerfiles already `apt/apk upgrade` ; rebuild monthly so the vuln scan stays clean).
- Re-run `npm run test:integration:setup` after schema/seed changes.
- Recommended ongoing support: bug fixes + minor features + dependency/security updates (typical 15–20%/yr of build cost).

9. Handover sign-off

Item	Confirmed
Source code delivered & pushed to <code>main</code>	☒
All automated tests + QA Robot green	☒
CI/CD green on <code>main</code>	☒
Documentation complete (guide, API, ops, deploy, threat model)	☒
No real PII / secrets in the repository	☒
Demo environment runnable from a clean checkout	☒
Outstanding (customer) items documented	☒

Delivered by: _____ Date: _____

Accepted by (AMW): _____ Date: _____

Document index

[Developer & Admin Guide](#) · [README](#) · [API](#) · [Data Model](#) · [Deploy Checklist](#) · [Operations Runbook](#) · [Production Readiness](#) · [System Test Cases](#) · [Threat Model](#) · [UX/UI Standards](#)